

A Comparison of Nearest Neighbour and Classification Tree Models for Network Intrusion Detection

M. Tukker

Stellenbosch University
Stellenbosch, South Africa
Student number: 25868640
Email: 25868640@sun.ac.za

Abstract—A network intrusion detection system assigns a network flow to a category of attack from the measured properties of that flow. The task combines a severely skewed distribution of the categories with a mixture of numeric and nominal features, and both properties bear on the choice of model. This study compared a k -nearest neighbour classifier with a classification tree on a dataset of 257 673 network flows described by 43 features and labelled with one of ten attack categories. The expected effect of each data quality issue of the dataset on each model was stated before any model was fitted. The pre-processing applied to each model followed from those expectations, so that each model received the transformations that the mechanism of that model required and no others. The nearest neighbour model received standardised numeric features, a one-hot encoding of the nominal features, and the removal of one member of each strongly correlated pair. The tree received the numeric features unaltered and an ordinal encoding of the nominal features. Neither model received resampled training data. Both models were tuned by grid search within a five-fold nested cross-validation over the complete dataset, and were compared on the macro-averaged F1 score. The classification tree attained a macro-averaged F1 score of 0.5791 against 0.4663 for the nearest neighbour model, and ranked higher on every fold. The advantage of the tree was concentrated in the rarest attack categories. Classification accuracy was found to be uninformative on this dataset, because both models exceeded an accuracy of 0.76 against a majority-class baseline of 0.3609 while failing on the categories that the task exists to detect.

Index Terms—imbalanced classification, k -nearest neighbours, classification trees, network intrusion detection

I. INTRODUCTION

A network intrusion detection system distinguishes an attack from ordinary traffic by the measured properties of a network flow. The volume of traffic on a modern network makes manual inspection impractical, so the task is posed as a supervised classification problem in which a model learns the mapping from the properties of a flow to a category of attack. The task is difficult for two reasons that are properties of the data rather than of any model. Attacks are rare relative to ordinary traffic, and the categories of attack differ greatly in frequency among themselves. A measured flow also mixes numeric quantities, such as a duration or a byte count, with nominal descriptions, such as a transaction protocol.

The goal of this study was to determine which of two machine learning models is better suited to the task, and to justify the determination from the mechanism of each model rather than from the performance figures alone. The two models considered were the k -nearest neighbour classifier and the classification tree. A dataset of 257 673 network flows, described by 43 features and labelled with one of ten attack categories, was used as the empirical basis [6].¹

The two models were treated separately throughout. The data quality issues of the dataset were examined first, and the expected effect of each issue on each model was stated before any model was fitted. The pre-processing applied to each model followed from those expectations, so that each model received the transformations that the mechanism of that model required and no others. Both models were then tuned by grid search within a nested cross-validation and compared on measures chosen for a skewed class distribution.

The classification tree outperformed the nearest neighbour model on every measure reported, and the advantage was concentrated in the rarest attack categories. The properties of the tree that account for the difference are the properties that allowed the tree to be fitted with the fewest transformations of the data.

Section II describes the two models and states the expected effect of each data quality issue on each model. Section III describes the experimental design. Section IV describes the pre-processing applied to each model, the tuning of the control parameters, the performance measures and the analysis procedure. Section V reports the results, and Section VI concludes.

II. BACKGROUND

Network intrusion detection is a supervised classification problem, in which a model learns a mapping from the measured properties of a network flow to an attack category. This section provides the background required for the remainder of the report. The section first describes the two machine learning models applied in this study, and then states the expected

¹The source code is available at <https://github.com/MikeTuks/Assignment-2>.

effect of each remaining data quality issue on each of the two models.

A. Machine Learning Algorithm Description

1) *Nearest Neighbour Classification*: The k -nearest neighbour classifier is an instance-based model, which stores the training instances and does all of its computation at prediction time [4]. No model is fitted during training, which is why the approach is described as lazy learning [5]. A prediction is produced by measuring the distance from the query instance to every stored instance. The query instance takes the class held by the majority of the k closest instances.

The distance measure defines the model. Euclidean distance is the usual choice for numeric features, and takes the square root of the sum of the squared differences between corresponding feature values. The measure treats every feature as an equally weighted coordinate. A feature measured on a wide numeric range therefore contributes more to the distance than a feature measured on a narrow range. The parameter k controls the smoothness of the decision boundary. The approach also degrades as the number of features grows [1], [2].

2) *Classification Trees*: A classification tree partitions the feature space by recursively splitting the training set on the values of a single feature [8]. Each internal node holds a test on one feature, and each leaf holds a class. A prediction is produced by following the tests from the root to a leaf, and returning the class held by that leaf.

The rule that selects the splitting feature controls how a tree is built. Entropy measures the impurity of a set of instances with respect to the class labels, and information gain measures the reduction in entropy achieved by a candidate split [8]. The feature with the greatest information gain is chosen. The Gini index provides an alternative impurity measure [5]. A tree grown until every leaf is pure describes the training set exactly and generalises poorly, because the deepest splits fit noise rather than structure [5]. Pruning addresses the overfitting by removing subtrees that do not improve performance on holdout data. Growth can also be constrained in advance by limiting the depth of the tree or by requiring a minimum number of instances in a leaf.

Two properties of the mechanism matter for the present study. A split compares values of a single feature against a threshold, so a tree responds to the rank order of the values of a feature and is unaffected by the scale on which the feature is measured. A tree also handles missing values without imputation [8].

B. Expectations With Respect to Data Quality Issues

This section states the expected effect of each remaining data quality issue on the two models, before any model is trained. The expectations follow from the mechanism of each algorithm. A k -nearest neighbour classifier assigns a class from the distance to stored instances, so every issue that distorts the distance function distorts the prediction [4]. A classification tree partitions the feature space one feature at a time [8]. Each split is scored by an impurity measure. A tree

TABLE I
EXPECTED IMPACT OF EACH DATA QUALITY ISSUE ON EACH MODEL

Issue	Nearest neighbours	Classification tree
Differing numeric ranges	Severe: widest range dominates the distance	None: splits use rank order
Large positive outliers	Distorted neighbourhoods	Minor: isolated then pruned
Correlated features	Duplicated information is double weighted	Minor: one feature per split
Missing values	Imputation or deletion, neither free	Tolerated by gain scaling
Nominal features	Encoding inflates the dimension count	None: split on level subsets
Unique identifier	Added noise dimension	Severe: memorises the training set
Skew class distribution	Minority classes out-voted	Minority partitions pruned away

therefore responds to the order of the values of a feature rather than to how large those values are. The same issue therefore carries a different cost for each model. Table I summarises the expectations discussed below.

1) *Differing Numeric Ranges*: The numeric features of the dataset are measured on widely different scales. The feature `sload` covers a range of 5.99×10^9 , whereas the binary feature `is_sm_ips_ports` covers a range of one. A Euclidean distance is a sum of squared coordinate differences, so the feature with the widest range dominates that sum. The expectation for the nearest neighbour model is severe degradation, because the neighbourhood of an instance is determined almost entirely by `sload`. The impurity of a split does not depend on the units of a feature. The expectation for the tree is therefore no measurable effect. Scaling is expected to be necessary for the nearest neighbour model and unnecessary for the tree.

2) *Large Positive Outliers*: Values beyond three standard deviations from the mean occur in 23.29 per cent of the instances on at least one feature. An outlier displaces an instance in the feature space, which enlarges the distance to that instance. The expectation for the nearest neighbour model is a distorted neighbourhood structure, compounded by the range disparity discussed above. A tree isolates an extreme value in a small partition, which pruning then absorbs into the parent partition. The extreme value remains a minority and does not change the majority class. The expectation for the tree is therefore robustness [5].

3) *Correlated Features*: Twelve pairs of features correlate above 0.95 in absolute value, as listed in Table II. The pair `is_ftp_login` and `ct_ftp_cmd` correlates at 0.999. Two features that carry the same information contribute that information twice to a distance sum. The expectation for the nearest neighbour model is a biased distance function that overstates the importance of the duplicated quantities. A tree selects one member of a correlated pair and gains little from the second member. The expectation for the tree is a marginal effect on accuracy. The choice of split feature is expected to be unstable, because near-identical features produce near-

TABLE II
STRONGLY CORRELATED NUMERIC FEATURE PAIRS

Feature	Correlated feature	$ r $
is_ftp_login	ct_ftp_cmd	0.999
dbytes	dloss	0.997
sbytes	sloss	0.996
swin	dwin	0.980
dpkts	dloss	0.980
ct_srv_src	ct_srv_dst	0.979
dpkts	dbytes	0.973
spkts	sloss	0.972
spkts	sbytes	0.964
ct_dst_ltm	ct_src_dport_ltm	0.962
ct_dst_src_ltm	ct_srv_dst	0.960
ct_srv_src	ct_dst_src_ltm	0.954

identical impurity scores.

4) *Missing Values*: The feature `service` is missing in 54.85 per cent of the instances, and no other feature contains missing values. A distance calculation requires a value for every coordinate, so the nearest neighbour model cannot proceed without either imputation or deletion. Imputing more than half of a feature invents the geometry of that dimension. Deletion discards the information carried by the 45.15 per cent of instances for which the value is present. The expectation for the nearest neighbour model is that neither option is free. A tree scales the information gain of a candidate split by the fraction of instances for which the value is known, and distributes an instance with a missing value across the outcomes of a split in proportion to the observed outcome frequencies [8]. The expectation for the tree is therefore tolerance of the missing values without imputation.

5) *Mixed Numerical and Categorical Features*: The features `proto`, `state` and `service` are nominal. The three features carry 133, 11 and 12 levels respectively. A distance function works on numeric coordinates, so a nominal feature needs an encoding before the nearest neighbour model can use the feature. A one-hot encoding of `proto` adds 133 sparse dimensions. Distances concentrate as the number of dimensions grows, which reduces the contrast between the nearest and the furthest neighbour [1], [2]. The expectation for the nearest neighbour model is a loss of discrimination that grows with the cardinality of the encoded feature. A tree splits a nominal feature on subsets of the levels directly and requires no encoding. The expectation for the tree is no penalty from the nominal features, subject to the many-values bias discussed below.

6) *Unique Identifier*: The feature `id` holds a distinct value for each of the 257 673 instances. Information gain favours a feature with many outcomes, because a split on such a feature produces small pure partitions [8]. A split on `id` produces one instance per partition and reduces impurity to zero, which makes `id` the highest-scoring candidate at the root. The resulting tree memorises the training set and generalises to no unseen instance. The expectation for the tree is therefore catastrophic failure. The failure is avoided by removing `id`. A gain ratio criterion offers an alternative remedy, and normalises

information gain by the entropy of the split itself. The nearest neighbour model treats `id` as one more noise coordinate. The expectation for the nearest neighbour model is therefore serious but not catastrophic degradation. Removal of `id` is expected to be necessary for both models.

7) *Skew Class Distribution*: The target feature is distributed across ten classes in a ratio of 534 to one between the largest and the smallest class. The Normal class holds 93 000 instances of the complete dataset and the Worms class holds 174 instances. A nearest neighbour vote is decided by which classes appear in the neighbourhood. The Worms class occupies 0.07 per cent of the dataset, and an instance of that class is therefore outvoted in almost every neighbourhood in which such an instance appears. The expectation for the nearest neighbour model is a recall near zero on the smallest classes, worsening as k grows. A tree forms small partitions for the minority classes, which pruning then removes. The expectation for the tree is therefore the same failure through a different mechanism [5]. The skew is also expected to make classification accuracy uninformative, because a classifier that predicts the majority class for every instance reaches an accuracy of 36.09 per cent while identifying no attack.

III. METHODOLOGY

The study was implemented in Python, using the scikit-learn machine learning library for both models and for the pre-processing.²

Each model was constructed as a pipeline, in which the pre-processing steps and the classifier form a single estimator. The construction matters for the validity of the results. A pipeline fits every pre-processing step on the training partition only, then applies that fitted step to the validation partition without refitting. Standardisation performed outside a pipeline computes the mean and the standard deviation over the complete dataset, which allows information from the validation partition to reach the training partition. Using a pipeline prevents that leakage.

The two pipelines share the structure described above. The pre-processing applied within each pipeline differs, as described in Section IV-A. Every model was fitted on all 257 673 instances of the dataset. No sub-sample was drawn. The nested cross-validation procedure described in Section IV-D fits a nearest neighbour model once for every combination of control parameter values, in every inner fold of every outer fold, and the cost of that procedure on the complete dataset is within the resources available. Retaining every instance preserves the 174 instances of the smallest class, which a sub-sample would reduce.

IV. EMPIRICAL PROCEDURE

This section describes the procedure followed to obtain and compare the performance of the two models. The section first describes the pre-processing applied to each model, and justifies every transformation and every omission. The tuning

²Python 3.10, scikit-learn 1.5.0, pandas 2.2.2 and NumPy 2.2.6.

TABLE III
PRE-PROCESSING STEPS APPLIED TO EACH MODEL

Step	Nearest neighbours	Classification tree
Removal of <code>id</code>	Applied	Applied
Standardisation	Applied	Not applied
Removal of correlated features	Applied	Not applied
One-hot encoding	Applied	Not applied
Ordinal encoding	Not applied	Applied
Imputation of <code>service</code>	Not applied	Not applied
Outlier removal	Not applied	Not applied
Resampling	Not applied	Not applied

of the control parameters follows. The performance measures are then defined and justified against the class distribution. The section closes with the cross-validation protocol under which the two models were compared.

A. Data Pre-Processing

Two separate pre-processing pipelines were constructed, one for each model. A single shared pipeline would apply transformations that one of the two models does not require, and an unnecessary transformation costs information without any compensating benefit. The expectations stated in Section II-B determine which transformations each model requires. Table III lists the steps applied to each model.

Two transformations were applied to both models. The feature `id` was removed, because a feature holding a distinct value for every instance carries no information that generalises beyond the training set. Retaining `id` would further supply the tree with a split that reduces impurity to zero without predictive value. The target feature required no transformation, because the target feature of the revised dataset contains no missing values and is already encoded as an integer.

Standardisation was applied to the numeric features of the nearest neighbour model alone. Standardisation transforms each feature to zero mean and unit variance, which equalises the contribution of every feature to the Euclidean distance. The transformation is needed because the numeric features are measured on widely different scales, as described in Section II-B. Without standardisation the distance is determined almost entirely by the feature with the widest range. Standardisation was deliberately not applied to the tree. A split compares values of a single feature against a threshold, so a monotone transformation of that feature changes neither the candidate thresholds nor the resulting partition. Standardising the input of the tree would add computation and hide the units of the split thresholds, without changing a single split.

Table II lists every pair of numeric features with an absolute Pearson correlation above 0.95, in descending order of correlation. The second feature of each pair was removed for the nearest neighbour model, which leaves no pair above 0.95 among the retained features. Nine features were removed on this basis, namely `sbytes`, `sloss`, `dbytes`, `dloss`, `dwin`, `ct_ftp_cmd`, `ct_srv_dst`, `ct_dst_src_ltm` and `ct_src_dport_ltm`. Two features that carry the same

information contribute that information twice to a distance sum, which weights the underlying quantity twice as heavily as an uncorrelated quantity. Removing the redundant members restores the intended weighting. The removal was deliberately not applied to the tree, because a tree selects one feature per split and gains nothing from the second member of a correlated pair. Removing a correlated feature from the input of the tree would discard a feature that the tree is free to select when the feature yields the greater information gain.

The nominal features were encoded differently for the two models. The nearest neighbour model requires numeric coordinates. The features `proto`, `service` and `state` were therefore one-hot encoded. Encoding all 133 protocol levels would add 133 sparse dimensions and worsen the concentration of distances described in Section II. Levels occurring in fewer than one per cent of the instances were therefore grouped into a single infrequent category, which retains the frequent protocols as distinct coordinates and collapses the long tail. A threshold of one per cent retains the five protocols that occur in more than one per cent of the instances, namely `tcp`, `udp`, `unas`, `arp` and `ospf`, and collapses the remaining 128 levels. A threshold of 1.5 per cent would discard `arp` and `ospf`, which occur in 3846 and 3271 instances respectively. The tree received an ordinal encoding, which maps each level to an integer and adds no dimensions. An ordinal encoding imposes an arbitrary order on the levels. The imposed order is a defect for a distance-based model but not for a tree, because a tree separates any subset of the integer codes through successive splits.

The missing values of `service` were not imputed for either model. The encoders treat the absent value as a category of the feature. Three observations support the decision. The feature is absent in 54.85 per cent of the instances, so imputation would invent most of the values of the feature. The absence is furthermore not random. The proportion of instances for which `service` is absent varies from 1.6 per cent of the Generic class to every instance of the Shellcode class, as reported in Table IV, so the absence carries information about the target feature. Imputation would replace that information with a predicted value.

The absence is also structural rather than accidental. A `service` is recorded for two of the 133 protocols, namely `tcp`

TABLE IV
ABSENCE OF `SERVICE` BY ATTACK CATEGORY

Attack category	Instances without a service (%)
Generic	1.6
Worms	14.9
Exploits	51.8
Normal	68.7
Analysis	78.9
Reconnaissance	84.2
DoS	84.7
Fuzzers	90.7
Backdoor	95.3
Shellcode	100.0

and udp. The remaining 131 protocols report no service for any instance. Protocols such as arp and ospf operate below the application layer and have no service to report, so no true value exists to recover. The accompanying feature description supplied with the dataset defines an unused service as a category of the feature, which confirms the reading. Treating the absence as a category therefore records a property of the flow, whereas imputation would assign an application-layer protocol to flows that carry none.

Two remaining issues were deliberately left unaddressed. The outliers were not removed or capped. The values concerned are large but legitimate measurements of network flows rather than recording errors, and an extreme flow size is plausibly the signal that distinguishes an attack from normal traffic. Removing the outliers risks discarding the evidence the models require. Standardisation moderates the effect of the outliers on the nearest neighbour model without discarding any instance. The class imbalance was not addressed by resampling. Synthetic minority over-sampling generates a new instance by interpolating between a minority instance and the nearest neighbours of that instance in the same class [3]. Chawla *et al.* applied the technique to two classes at a time, and reduced a seven-class dataset to a pair of classes for that reason [3]. The target feature of the present dataset holds ten classes, so the technique would have to be extended beyond the setting in which the technique was evaluated. The interpolation is also unreliable at the class size that matters here. The Worms class holds 174 instances of the 257 673 instances of the dataset. A synthetic instance for that class would be interpolated between neighbours drawn from those 174 instances, which are themselves distant in a space of 48 coordinates, for the reason given in Section II. Resampling further alters the class distribution that the models encounter, which would confound the comparison between the two models with the effect of the resampling procedure. The imbalance was instead addressed through the choice of performance measure described in Section IV-C, and through the class weighting offered to the tree as a control parameter.

TABLE V
CONTROL PARAMETER VALUES SEARCHED FOR EACH MODEL

Model	Parameter	Values searched
Nearest neighbours	Neighbours	1, 5, 11, 25
	Weighting	uniform, distance
Classification tree	Criterion	Gini index, entropy
	Maximum depth	unlimited, 10, 20
	Minimum leaf size	1, 5, 20
	Class weighting	none, balanced

B. Control Parameter Tuning

The control parameters of both models were selected by exhaustive grid search over the ranges listed in Table V. The search was conducted independently within every outer fold of the cross-validation procedure, so the configuration selected

for one fold does not depend on the data against which that configuration is tested.

The grid for the nearest neighbour model covers the number of neighbours and the weighting of the neighbour votes. Four values of the number of neighbours were searched, ranging from a single neighbour to 25 neighbours. At the lower end of the range the decision boundary follows the training data exactly. At the upper end the boundary is smoothed over a neighbourhood large enough to suppress the smallest classes. The four values are odd, so that a vote between two classes cannot tie. An odd number of neighbours does not exclude a tie among the ten classes of the target feature, and the distance weighting described below removes the remaining ties. Both uniform and distance weighting were searched, because distance weighting allows a close neighbour of a minority class to outweigh several distant neighbours of the majority class.

The grid for the tree covers the impurity criterion, the maximum depth, the minimum number of instances in a leaf, and the class weighting. The Gini index and entropy were both searched, because neither criterion is uniformly superior. Three values of the maximum depth and three values of the minimum leaf size were searched, which together control the extent to which the tree is permitted to fit the training data. The balanced class weighting was included as a candidate. The weighting rescales the contribution of each class to the impurity measure in inverse proportion to the frequency of that class. The weighting therefore gives the tree a defence against the class skew that the nearest neighbour model does not have.

The grid search was conducted over a three-fold inner cross-validation, and the configuration attaining the greatest macro-averaged F1 score was retained. The number of independent runs is five for each model, one for each outer fold. Every result reported in Section V is the mean over those five runs, accompanied by the standard deviation.

C. Performance Metrics

Classification accuracy is the proportion of instances assigned to the correct class. Accuracy is a poor measure on this dataset. The class distribution is skewed in a ratio of 534 to one, and a classifier that assigns every instance to the Normal class attains an accuracy of 0.3609 while identifying no attack at all. A measure that weights every instance equally therefore rewards a classifier that ignores the rare attack categories, which are the categories the task exists to detect.

The macro-averaged F1 score was adopted as the primary measure. The F1 score of a class is the harmonic mean of the precision and the recall of that class. Precision is the proportion of instances assigned to a class that belong to that class. Recall is the proportion of instances of a class that are assigned to that class. The macro average is the unweighted mean of the F1 scores over the ten classes, so the Worms class contributes as much to the reported figure as the Normal class despite holding 174 instances against 93 000. The measure

penalises a classifier that ignores a rare class, which is the behaviour that accuracy rewards [9].

Balanced accuracy was adopted as a secondary measure, and is the unweighted mean of the recall over the ten classes. The measure disregards false alarms, and therefore reports the proportion of each attack category that the classifier detects without reference to the cost of the incorrect alarms raised. Balanced accuracy read together with the macro-averaged precision separates the two failure modes. A classifier that detects few attacks and a classifier that raises many false alarms are distinguished by the two measures, but not by the F1 score alone.

Accuracy and the support-weighted F1 score are additionally reported against the accuracy of the majority-class classifier, to show the margin by which a skew-blind measure overstates performance on this dataset.

D. Analysis Process

The two models were compared under nested cross-validation. The outer loop partitions the dataset into five folds, and reports the performance of each model on each fold in turn after fitting on the remaining four. The inner loop partitions each training partition into a further three folds, over which the grid search of Section IV-B selects the control parameters. The nested arrangement is needed because tuning and testing on the same data gives an over-optimistic estimate of performance. No outer fold influences the configuration that is tested on that fold.

Five outer folds were used rather than a larger number, for two reasons. The number of folds governs the size of the test partition, and five folds place approximately 51 500 instances in each test partition. The smallest class holds 174 instances, so a test partition of that size is expected to contain approximately 35 instances of the Worms class. A larger number of folds would reduce that count and render the per-class measures for the rarest classes unstable. The number of folds further multiplies the cost of the nested procedure, and five folds bound the runtime of the search described in Section IV-B within the resources available.

Every partition was stratified, so that each fold preserves the class proportions of the dataset. The choice is not optional on this dataset. A partition drawn at random from a dataset in which the smallest class holds 174 of 257 673 instances is liable to produce a fold holding no instance of that class. The recall of the class is undefined for such a fold. The same random seed was used for every partition, so that both models were fitted and tested on identical folds and the comparison between the two models is paired.

The difference between the two models was tested with a paired t -test over the five folds. The tree attained the greater macro-averaged F1 score on every fold, and the paired t -statistic is 22.56 on four degrees of freedom ($p = 2.3 \times 10^{-5}$). The test is reported but not relied upon. The training partitions of a five-fold procedure share three quarters of their instances, so the folds are not independent and the paired t -test understates the variance of the difference. Nadeau and

Bengio [7] proposed a corrected statistic that inflates the variance to account for that overlap, which gives $t = 15.04$ and $p = 1.1 \times 10^{-4}$ and leaves the conclusion unchanged.

V. RESEARCH RESULTS

The two models were compared under the nested cross-validation procedure described in Section IV-D. This section reports the performance of each model, identifies the classes on which the models differ, and relates the observed performance to the expectations stated in Section II-B.

Table VI reports the mean and the standard deviation of each measure over the five outer folds. The classification tree outperformed the nearest neighbour model on every measure. The tree attained a macro-averaged F1 score of 0.5791 against 0.4663 for the nearest neighbour model. The standard deviations of the two scores are 0.0062 and 0.0057, so the difference of 0.1128 is roughly eighteen times the between-fold standard deviation. The ranking of the two models is therefore consistent across every fold and does not depend on the partition.

The accuracy figures demonstrate the argument of Section IV-C. Both models attain an accuracy above 0.76, which exceeds the majority-class baseline of 0.3609 and suggests competent performance. The macro-averaged measures of the same models are 0.4663 and 0.5791. The gap between the two families of measure is the contribution of the rare classes, which the accuracy figure conceals and the macro average exposes.

A. Performance by Class

Table VII reports the F1 score of each model on each class, measured on a stratified holdout partition of 51 535 instances under the configuration selected in the majority of folds. The two models agree on the classes at the extremes. Both identify the Generic class almost perfectly, at 0.99, and both identify the Normal class well. Both fail on the Backdoor class and on the Analysis class, at 0.15 or below.

The advantage of the tree is concentrated in the rare classes. The tree scored 0.58 on the Worms class against 0.30 for the nearest neighbour model, and 0.61 on the Shellcode class against 0.35. The Worms class holds 35 instances of the holdout partition, and the recall of the nearest neighbour model on that class was 0.20. The expectation stated in Section II-B was a recall near zero on the smallest classes

TABLE VI
PERFORMANCE UNDER FIVE-FOLD NESTED CROSS-VALIDATION

Measure	k -Nearest Neighbours		Classification Tree	
	Mean	Std. dev.	Mean	Std. dev.
Macro-averaged F1	0.4663	0.0057	0.5791	0.0062
Balanced accuracy	0.4475	0.0049	0.5638	0.0073
Macro-averaged precision	0.5248	0.0060	0.6557	0.0075
Macro-averaged recall	0.4475	0.0049	0.5638	0.0073
Weighted F1	0.7602	0.0015	0.8053	0.0014
Accuracy	0.7647	0.0017	0.8102	0.0030

TABLE VII
F1 SCORE BY CLASS ON THE HOLDOUT PARTITION

Class	Instances	k -NN	Tree
Generic	11 774	0.99	0.99
Normal	18 600	0.88	0.91
Exploits	8 905	0.66	0.69
Reconnaissance	2 798	0.59	0.83
Fuzzers	4 849	0.48	0.60
DoS	3 271	0.36	0.32
Shellcode	302	0.35	0.61
Worms	35	0.30	0.58
Analysis	535	0.15	0.15
Backdoor	466	0.02	0.14

for the nearest neighbour model, and the observed recall supports that expectation. A minority instance is outvoted in the neighbourhood in which the instance appears, whereas a tree isolates a small pure region through successive splits and predicts the minority class within that region.

The DoS class is the one class on which the nearest neighbour model scored higher, at 0.36 against 0.32. Table VIII attributes the behaviour of both models on that class to the Exploits class. The tree assigned 2 074 of the 3 271 DoS instances to the Exploits class. The tree assigned only 1 036 of those instances to the DoS class. A DoS flow is therefore more likely to be labelled an exploit than a denial of service by the tree. The nearest neighbour model divided the DoS instances more evenly between the two classes, at 1 542 against 1 279. The more even division accounts for the higher F1 score of the nearest neighbour model on this one class. Both models also assigned a substantial number of Exploits instances to the DoS class. The two categories are not separable in the measured features. The inseparability is a property of the dataset rather than of either model.

TABLE VIII
CONFUSION BETWEEN THE DoS AND EXPLOITS CLASSES

True class	k -Nearest Neighbours		Classification Tree	
	DoS	Exploits	DoS	Exploits
DoS (3 271)	1 279	1 542	1 036	2 074
Exploits (8 905)	1 712	5 956	1 482	6 746

B. Selected Control Parameters

The nearest neighbour model selected five neighbours with distance weighting in every one of the five folds. The agreement across folds indicates that the choice is determined by the data rather than by the partition. Distance weighting was selected in preference to uniform weighting in every fold, which is consistent with the expectation that a minority instance is outvoted under uniform weighting.

The tree selected an unlimited depth in four of the five folds, the entropy criterion in four of the five folds, and a minimum leaf size of five instances in three of the five folds. The variation between folds is confined to the parameters that control the size of the tree. The resulting scores vary by less than one standard deviation, so the disagreement is not

material. Balanced class weighting was available to the tree as a control parameter and was rejected in every fold. Weighting the classes raises the recall of the rare classes at a cost in precision that the macro-averaged F1 score does not repay. The result supports the decision recorded in Section IV-A not to resample the training data, because the one reweighting mechanism that was offered to either model was not selected.

C. Interpretation

The results are consistent with the expectations stated in Section II-B. Both models were expected to degrade on the rare classes, and both did. The tree was expected to be the more robust of the two models under the data quality issues of the dataset. A tree is unaffected by the scale of a feature. A tree admits an ordinal encoding of a nominal feature without adding dimensions. A tree also treats an absent value as a category. Each of those properties removes a transformation that the nearest neighbour model required, and each transformation is a further opportunity for information to be lost.

The performance of the nearest neighbour model is further limited by the dimensionality of the encoded feature space. The model measures distance over 48 coordinates, of which 18 arise from the one-hot encoding of the three nominal features. The concentration of distances in a space of that dimensionality reduces the contrast between the nearest and the furthest neighbour of an instance [1], [2], and the effect falls hardest on the classes for which few instances are available.

The results would be expected to change under two circumstances. A distance measure that weights the nominal features separately from the numeric features, such as a measure defined for mixed data, would address the dimensionality of the encoding rather than the symptom. A cost matrix stating the relative cost of a missed attack against the cost of a false alarm would also change the ranking. The macro-averaged F1 score weights every class equally, which is an assumption about the operational cost rather than a measurement of that cost.

VI. CONCLUSIONS

This study compared a k -nearest neighbour model and a classification tree on the task of assigning a network flow to one of ten attack categories. Each model received the pre-processing that the mechanism of that model required, and no more. The nearest neighbour model received standardised numeric features, a one-hot encoding of the three nominal features, and the removal of one member of each strongly correlated pair. The tree received the numeric features unaltered and an ordinal encoding of the nominal features. Both models were tuned by grid search within a five-fold nested cross-validation over all 257 673 instances of the dataset, and were compared on the macro-averaged F1 score.

The classification tree was the better of the two approaches for this problem. The tree attained a macro-averaged F1 score of 0.5791 against 0.4663 for the nearest neighbour model,

and ranked higher on every fold. The advantage of the tree was concentrated in the rare attack categories, which are the categories that the task exists to detect. Three properties of the tree accounted for the difference. A tree is unaffected by the scale on which a feature is measured, admits a nominal feature without adding a coordinate to a distance calculation, and treats an absent value as a category rather than as a quantity to be imputed. Each property removed a transformation that the nearest neighbour model required, and each such transformation was an opportunity for information to be lost.

Two observations were drawn from the empirical process. The first is that classification accuracy was uninformative on this dataset. Both models attained an accuracy above 0.76 against a majority-class baseline of 0.3609, while the macro-averaged scores of the same models revealed a failure on the rarest categories that the accuracy figure concealed. The second is that the class imbalance was addressed more effectively through the choice of performance measure than through the reweighting of the classes, because balanced class weighting was offered to the tree as a control parameter and was rejected in every fold.

Three directions extend the present study. A distance measure defined for mixed numeric and nominal data would address the dimensionality of the encoded feature space directly, rather than through the removal of correlated features. A cost matrix stating the relative cost of an undetected attack against the cost of a false alarm would replace the equal weighting of the classes that the macro average assumes. The confusion between the denial of service and Exploits categories accounted for 27 per cent of the errors of the nearest neighbour model and 36 per cent of the errors of the tree. Additional features that separate the two categories would therefore address the largest single source of error in both models.

The study showed that the choice between the two models followed from the data quality issues of the dataset rather than from the models alone. The transformations that each model required were determined before any model was fitted, and the model that required the fewest transformations attained the greater performance.

REFERENCES

- [1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim, "On the surprising behavior of distance metrics in high dimensional space," in *Proceedings of the Eighth International Conference on Database Theory*, 2001, pp. 420–434.
- [2] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, "When is nearest neighbor meaningful?" in *Proceedings of the Seventh International Conference on Database Theory*, 1999, pp. 217–235.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [4] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [5] J. D. Kelleher, B. M. Namee, and A. D'Arcy, *Fundamentals of Machine Learning for Predictive Data Analytics*. Cambridge, Massachusetts: The MIT Press, 2015.
- [6] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proceedings of the Military Communications and Information Systems Conference*, Canberra, Australia, 2015, pp. 1–6.
- [7] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.
- [8] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [9] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing and Management*, vol. 45, no. 4, pp. 427–437, 2009.